

PL/SQL Interview Scenarios – Complete Answer Guide

This guide includes complete answers and explanations for 10 real-world PL/SQL interview scenarios.

Each problem is practical, scenario-based and designed to test your ability to think like an Oracle developer.

1 Track Data Archival

```
CREATE OR REPLACE PROCEDURE archive_old_orders IS  
BEGIN  
    INSERT INTO archived_orders  
    SELECT * FROM orders WHERE order_date <  
    ADD_MONTHS(SYSDATE, -36);  
  
    DELETE FROM orders WHERE order_date <  
    ADD_MONTHS(SYSDATE, -36);  
END;
```

■ Explanation: Moves orders older than 3 years to a separate table and then deletes them from the main one. Ensures performance by keeping the working table lighter.

2 Dynamic Table Update

```
CREATE OR REPLACE PROCEDURE update_status(  
    p_table_name IN VARCHAR2,  
    p_status IN VARCHAR2  
) IS  
    v_sql VARCHAR2(1000);  
BEGIN  
    v_sql := 'UPDATE ' || p_table_name || ' SET status = :1';  
    EXECUTE IMMEDIATE v_sql USING p_status;  
END;
```

■ Explanation: Uses dynamic SQL to update any table's status field. It allows reusable and generic logic across modules.

3 Send Alert on High Salary

```
CREATE OR REPLACE TRIGGER alert_high_salary
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
    IF :NEW.salary > 100000 THEN
        DBMS_OUTPUT.PUT_LINE('High salary alert: ' || :NEW.salary);
    END IF;
END;
```

■ Explanation: Triggers when salary exceeds threshold. In real-world cases, alerts can be email notifications or audit logs.

4 Auto Update Modified Date

```
CREATE OR REPLACE TRIGGER auto_modified_date
BEFORE UPDATE ON employees
FOR EACH ROW
BEGIN
    :NEW.last_modified := SYSDATE;
END;
```

■ Explanation: Automatically updates a timestamp field on every update. Common in audit-ready systems.

5 Email Validation Function

```
CREATE OR REPLACE FUNCTION is_valid_email(p_email IN
VARCHAR2) RETURN BOOLEAN IS
BEGIN
    RETURN INSTR(p_email, '@') > 1 AND INSTR(p_email, '.',
INSTR(p_email, '@')) > 0;
END;
```

■ Explanation: Basic check to ensure email contains '@' and a '.' after it. Can be expanded with regex for better accuracy.

6 Daily Transaction Summary

```
CREATE OR REPLACE PROCEDURE summarize_transactions IS
BEGIN
    INSERT INTO transaction_summary(summary_date, total_amount)
    SELECT TRUNC(SYSDATE), SUM(amount) FROM transactions;
END;

/* Schedule with */
BEGIN
    DBMS_SCHEDULER.create_job (
        job_name      => 'daily_summary_job',
        job_type       => 'PLSQL_BLOCK',
        job_action      => 'BEGIN summarize_transactions; END;',
        start_date      => SYSTIMESTAMP,
        repeat_interval => 'FREQ=DAILY; BYHOUR=23; BYMINUTE=59',
        enabled         => TRUE
    );
END;
```

■ Explanation: Logs the daily sum of transactions using a scheduled job. Useful in dashboards, reporting, or reconciliation processes.

7 Fetch Top Performer per Department

```
CREATE OR REPLACE FUNCTION get_top_employee(p_dept_id IN
NUMBER)
RETURN VARCHAR2 IS
    v_name VARCHAR2(100);
BEGIN
    SELECT emp_name INTO v_name
    FROM (
        SELECT emp_name FROM employees
        WHERE dept_id = p_dept_id
        ORDER BY performance_score DESC
    ) WHERE ROWNUM = 1;

    RETURN v_name;
END;
```

■ Explanation: Retrieves the top performer by score using ROWNUM and ORDER BY. Useful for reward systems or performance tracking.

8 Null Field Logger

```
CREATE OR REPLACE TRIGGER log_null_critical_fields
AFTER UPDATE ON users
FOR EACH ROW
BEGIN
    IF :NEW.email IS NULL AND :OLD.email IS NOT NULL THEN
        INSERT INTO null_field_log(field_name, user_id, log_time)
        VALUES ('email', :OLD.user_id, SYSDATE);
    END IF;
END;
```

■ Explanation: Tracks if key fields like email or phone are erased (set to NULL). Crucial for data integrity and compliance monitoring.

9 Temporary Lock for Failed Payment

```
DECLARE
    v_fail_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_fail_count
    FROM payments
    WHERE user_id = 101
    AND status = 'FAILED'
    AND payment_time >= SYSDATE - 1/24;

    IF v_fail_count >= 3 THEN
        UPDATE users SET lock_until = SYSDATE + (30/1440)
        WHERE user_id = 101;
    END IF;
END;
```

■ Explanation: Counts recent failures within the last hour and locks the user for 30 minutes. Prevents fraud or repeated invalid attempts.

10 Reopen Recently Closed Ticket

```
CREATE OR REPLACE PROCEDURE reopen_ticket(p_ticket_id IN  
NUMBER) IS  
    v_closed_time DATE;  
BEGIN  
    SELECT closed_at INTO v_closed_time FROM support_tickets WHERE  
ticket_id = p_ticket_id;  
  
    IF v_closed_time >= SYSDATE - 1 THEN  
        UPDATE support_tickets SET status = 'REOPENED' WHERE  
ticket_id = p_ticket_id;  
    END IF;  
END;
```

■ Explanation: Reopens tickets closed within the last 24 hours upon user action. Prevents hard closures for unresolved queries.